

Autonomous Maze Exploration

Information

I have set up the project with Claude code. Integrating an existing SLAM algorithm has not worked (BreezySLAM), so I decided to code a simplified SLAM algorithm on my own with Claude Code. The most lines of code were written by Claude code, some of them I refactored by myself and some method I too. I have not used any other code sources.

The code appears somewhat cluttered; a proper class design would have been too time-consuming in retrospect, but this could be implemented in the future. Overall, the approach works quite well, but there are still some issues with flying wall detection and the RGB-D camera. Mazes 2-5 were solved, but maze 1 presented problems reaching the blue cylinder due to the internal noisy map.

Maze Time Overview

Maze 1

Goal	Time
Blue cylinder reached	
Yellow cylinder reached	

Maze 2

Goal	Time
Blue cylinder reached	
Yellow cylinder reached	

Maze 3

Goal	Time
Blue cylinder reached	
Yellow cylinder reached	

Maze 4

Goal	Time
Blue cylinder reached	
Yellow cylinder reached	

Maze 5

Goal	Time
Blue cylinder reached	
Yellow cylinder reached	

1. General Concept

The central idea is that the robot incrementally builds a probabilistic, two-dimensional map of its surroundings while it drives, and repeatedly selects the nearest boundary between known and unknown space, called a frontier, as its next driving target. Exploration continues until the entire reachable area has been mapped or the Goals, reaching the blue and the yellow cylinder are reached.

First the robot turns around its own axes to scan the environment for the cylinders. After that, the mission automatically continues into a direct search phase: the robot looks for a blue and a yellow target object using its camera and, once each object has been located and confirmed as reachable, the robot drives to it.

The map itself is represented as a log-odds occupancy grid, a standard probabilistic mapping technique. Every cell stores a single scalar value expressing how confident the system currently is that the cell is occupied. Each new sensor reading updates this value by simple addition, the log-odds form of Bayes' theorem, so that repeated observations reinforce or erode a cell's occupancy belief over time instead of overwriting it outright. A cell only counts as a confirmed wall, or as confirmed free space, once enough consistent evidence has accumulated; until then it is treated as unknown and pulls the robot toward it during exploration.

Several independent sensors contribute evidence to this map. The lidar produces the primary wall layer. The RGB-D camera contributes three additional, functionally separate layers on top of it: a green floor-hazard layer, a low lidar-blind or high lidar-blind obstacle layer (flying walls), and a colour-tracking layer for the two target objects. These layers are kept deliberately separate rather than merged into a single grid, because the different sensors can disagree about the same physical location, and each layer needs to be able to correct itself independently without silently overwriting another sensor's evidence.

2. System Architecture and Classes

The software is organised as a set of small, single-purpose classes rather than one large controller. Each class owns exactly one responsibility – reading a sensor, maintaining the map, planning a path, or driving the wheels – and the top-level orchestrator wires them together. This keeps every sensor-specific quirk isolated in its own file instead of spreading it through the whole codebase.

Every simulation step the orchestrator performs the same three actions, independent of which high-level mission phase is currently active: it perceives (reads all sensors and updates the map), it acts (computes the next wheel command from the active state machine), and it renders (refreshes the live map visualisation). Perception never stops, even while the robot is busy driving toward a located target, so the map keeps improving throughout the whole mission.

The table below summarises the main classes and the file each of them lives in.

Class	File	Responsibility
Robot	robot.py	Hardware abstraction layer. Wraps the Webots device API (drive motors, wheel encoders, IMU, lidar, RGB-D camera, IR range sensors) into a small, robot-specific interface so every other

		module only ever talks to this class, never to Webots directly.
Odometry	odometry.py	Estimates the robot pose (x, y, heading) by integrating wheel-encoder distance deltas together with the IMU-measured heading.
OccupancyGrid	occupancy_grid.py	The central map data structure: a two-dimensional log-odds probability grid. It holds several independent layers -- the lidar wall map, a sticky floor-hazard mask, a camera-only low-obstacle map, and one color log-odds map per tracked object -- and exposes the Bayesian update and ray-tracing logic every sensor uses to write into it.
FrontierDetector	frontier.py	Finds “frontier” cells, i.e. reachable free cells that border unexplored space, and groups neighboring frontier cells into clusters that the planner can target.
PathPlanner	planner.py	Inflates obstacles by the robot’s radius, performs a flood fill to determine which cells are currently reachable, runs A* to plan a path, and scores candidate frontier clusters to pick the next exploration target.
Pilot	pilot_2.py	Converts a planned path into wheel velocity commands using pure-pursuit path following, together with a reactive safety layer that reads the short-range IR sensors directly.
MapViz	mapviz.py	Live matplotlib visualisation of the occupancy grid, the robot pose, and the current path, used for debugging while the simulation runs.
FloorHazardDetector	floor_hazard.py	Uses the RGB-D camera to find green floor tiles that must never be driven over and reports them as world-frame points.
ColorObjectDetector / TrackedObject	colored_objects.py	Detects the blue and yellow target objects in the RGB-D camera image and maintains one persistent, self-correcting position estimate per colour.
DepthObstacleDetector	depth_obstacle.py	Uses depth information alone (no colour) to find low obstacles that sit entirely above or below the lidar’s single fixed scanning height and would otherwise be invisible to the map.
CameraIntrinsics / camera_geometry.py	camera_geometry.py	Shared pinhole-camera mathematics (back-projection, RGB/depth registration, coordinate transforms) used identically by all three camera-based detectors above.
Explorer	explorer.py	Finite-state machine for pure frontier exploration: seeds the map with an initial spin, plans a path to the best frontier, drives it, and recovers from getting stuck.
GoToPoint	explorer.py	A simpler finite-state machine that drives the robot to one known, fixed target point (used once a tracked object has been located).

Mission	mission.py	Named string constants for the top-level mission state machine (EXPLORE_MAP, SEARCH_BLUE, GO_BLUE, SEARCH_YELLOW, GO_YELLOW, DONE, ...).
---------	------------	--

At the top of this structure, two finite-state machines share the driving duty. The Explorer state machine handles pure frontier exploration: it seeds the map with an initial full rotation, repeatedly plans a path to the best available frontier and drives it, and falls back to a short reverse-and-replan manoeuvre whenever the robot makes no measurable progress for too long. The GoToPoint state machine is structurally similar but drives to one single, already-known target point instead of choosing among frontiers, and is used once a tracked coloured object has been located. The MazeExplorer class owns both state machines, together with every sensor and map class, and switches between them according to the overall Mission state (map exploration, searching for blue, driving to blue, searching for yellow, driving to yellow, done).

3. Lidar-Based Mapping

The simulated lidar is modelled on the RPLidar A2, a genuinely rotating 2-D scanner that completes one full 360-degree sweep. Every control step, the robot reads back a full array of measured distances together with a matching, precomputed array of per-ray bearing angles, expressed in the robot's own reference frame.

Each lidar scan is fused into the occupancy grid using a classic inverse sensor model. For every individual ray, a Bresenham line is traced on the grid from the sensor origin to the measured hit point. All cells the line passes through before the hit are assumed to be free and have their log-odds value reduced; the single cell at the hit point is assumed to be occupied and has its log-odds value increased. Because the log-odds update is a simple addition, a cell that is scanned as free or occupied many times over successive frames accumulates strong evidence in one direction, while an occasional erroneous single reading is not enough to flip its classification. Rays that report no return, or a return beyond the configured usable range, are skipped entirely rather than treated as evidence of open space, since a lost beam gives no reliable information either way.

Because the lidar is a real rotating sensor rather than an instantaneous one, the whole range array returned by a single read is, strictly speaking, not captured at one exact heading: points at opposite ends of the array were measured up to one full sweep period apart. While the robot drives in a straight line this makes almost no difference, but while it is turning quickly the resulting heading drift can shift a wall's computed position by several centimetres, enough to erase an already-mapped wall cell with a mis-angled free-space ray. To avoid this, a scan is only integrated into the map while the robot's angular velocity stays below a configured threshold; scans captured while turning too fast are discarded for mapping purposes, although the robot's pose and mission logic continue normally regardless.

The resulting lidar wall map is treated as the most reliable source of truth about wall positions in the whole system. Several of the camera-based detectors described below explicitly check their own candidate detections against this map, either to avoid duplicating a wall the lidar has already mapped, or to reject a camera reading that would be physically impossible given a wall the lidar has already confirmed between the camera and the candidate point.

4. Floor Hazard Detection

Certain floor tiles in the maze are marked green and must never be driven over. The floor hazard detector scans the RGB-D camera image every few control steps, identifies every pixel that is simultaneously part of the floor plane and green in colour, and converts the surviving pixels into world-frame coordinates that the orchestrator then marks as permanent obstacles on the map – handled by the planner exactly like an ordinary wall, so no other part of the system needs to know that green hazard tiles exist as a separate concept.

A key technical difficulty this detector has to solve is that the RGB-D camera is really two separate lenses, a colour lens and a depth lens, mounted a few centimetres apart. Because of this physical offset, the same real-world point projects onto different pixel coordinates in the two images; naively reading the colour and the depth image at the same pixel coordinates therefore samples the colour of the wrong point, an error of roughly two dozen pixels at typical floor distances. This is corrected through proper RGB-D registration: each depth pixel is first back-projected into a three-dimensional point in the depth camera's own local frame using the pinhole camera model, that point is shifted by the known physical baseline between the two lenses to express it in the colour camera's frame instead, and it is then re-projected into the colour image to find the pixel that genuinely shows the same physical point.

Once a correctly registered three-dimensional point is available, two independent tests decide whether it is a hazard. The ground-plane test rotates the point by the camera's known mounting tilt and compares its computed height above the floor against a small tolerance, keeping only points that genuinely lie flat on the ground and rejecting anything that merely happens to be green while mounted on a wall. The colour test converts the registered pixel to hue, saturation and value and checks the hue channel specifically, rather than raw red-green-blue values, because hue alone identifies green reliably regardless of how brightly or dimly the tile happens to be lit.

Because a false negative here would let the robot drive over a genuine hazard while a false positive only costs a longer detour, hazard cells are marked permanently once detected: unlike the ordinary log-odds wall map, a hazard flag is never allowed to decay away again.

5. Coloured Object Detection and Tracking

A blue and a yellow object are placed somewhere in the maze and must be located and physically reached. Their detection reuses the exact same RGB-D registration pipeline as the floor hazard detector, but with the ground-plane test inverted: a candidate pixel only counts as a hit if it lies clearly above the floor, since the target objects are upright items rather than floor markings, and the whole image frame is scanned rather than only the area below the horizon, because an object can appear at any height in the picture.

Each colour is tracked with its own log-odds map, updated using exactly the same Bayesian logic as the lidar wall map: every camera frame that matches the colour adds positive evidence to the corresponding cell, and every frame that clearly shows a different colour at a location previously believed to hold the object adds negative evidence there instead. A false detection, for example a shadow or a similarly coloured surface seen once from an unlucky angle, therefore fades out again on its own the next time the camera looks at the same spot and disagrees, without any special-case handling.

A particular difficulty arises once the robot gets close to a target object: the depth camera has a minimum usable range and simply returns an invalid reading for anything closer than roughly sixty centimetres, which is exactly the range at which the object itself starts to fill the frame. Without further handling, this makes the object's own nearest pixels disappear from the depth image right when the robot needs them most, leaving only whatever the camera can still see peeking around or behind the object, typically the wall behind it, to be coloured instead. The detector compensates for this using the lidar as a fallback: for every pixel with

an invalid depth reading, it looks up the lidar's own measured range at the same bearing, and if the lidar confirms that something genuinely sits within the camera's blind zone in that direction, that range is substituted for the pixel before the normal colour test runs. This recovers the object's real near-field surface instead of only ever tagging whatever is visible behind it.

A tracked object's reported position is not stored as a running average but is instead recomputed, on demand, as the current centroid of every grid cell whose log-odds value for that colour is currently above threshold. This makes the position estimate self-correcting in exactly the same way the underlying log-odds map already is: as soon as a false-positive cell decays back below threshold it stops contributing to the centroid, and the reported position shifts back onto the real object on its own. A dedicated periodic cleanup pass additionally removes two further classes of false detection from each colour's map: very small clusters of only one or two cells, which are treated as camera noise, and clusters that turn out to be fused into the much larger connected wall network the lidar has mapped, which indicates the colour was actually picked up on a wall rather than on the standalone target object.

6. Depth-Based Obstacle Detection

The lidar only ever sweeps a single, fixed height above the floor. Any obstacle that sits entirely above or below that exact height, such as a low kerb, a raised sill, or an overhanging ledge, is completely invisible to it: the beam passes cleanly over or under the obstacle, the map shows free space there, and the robot would otherwise drive straight into something it never detected. The depth obstacle detector exists specifically to catch this class of lidar-blind obstacle using the RGB-D camera's depth channel alone, without relying on colour, since such obstacles may have no distinctive colour at all.

The detection works column by column in the depth image. An upright, camera-facing surface produces a vertical run of near-constant depth within a single image column, because every point on that face sits at roughly the same distance from the camera; the floor, by contrast, produces smoothly changing depth from one row to the next due to ordinary perspective. Grouping consecutive pixels of near-equal depth within a column therefore reliably flags candidate upright surfaces, while the ordinary floor is naturally excluded by this grouping step on its own.

Deciding whether a candidate run is actually obstacle-sized is done using its real, physical height rather than its raw pixel count. Both the run's topmost and bottommost pixels are back-projected into real-world height above the ground, and the difference between the two gives the run's true vertical extent in metres. A fixed pixel-count threshold would not work here, because the same physical obstacle occupies far more pixels in the image the closer the robot gets to it; a threshold tuned to correctly recognise an obstacle from two to four metres away would wrongly reject the very same obstacle as "too tall" once the robot closes to about one metre, purely as an artefact of perspective and with no real change in the obstacle itself.

A run that is both tall and grounded, meaning its lowest point sits at floor height, is treated as an ordinary wall rather than a lidar-blind obstacle: a surface that tall and that firmly attached to the floor necessarily also crosses the lidar's own scanning height somewhere along its length, so the lidar can be trusted to have already mapped it, and duplicating it through the camera pipeline is avoided. A run that is tall but does not reach the floor, in other words a genuinely suspended obstacle, is kept instead, since that is precisely the situation the lidar cannot see on its own. Short runs skip this distinction entirely, since whether a low kerb or step happens to touch the floor or float a few centimetres above it makes no practical difference to whether the robot can safely pass.

Regardless of height, every surviving candidate is finally checked against whether the robot can physically drive underneath it: the run's lowest visible point is back-projected to a real height above the ground, and if even that lowest point already clears the robot's physical clearance, the object is a hanging structure the robot simply passes under and is not marked as a wall. Only obstacles that would genuinely collide with the robot's body are kept.

Accepted candidates are reduced to a distance-and-bearing pair, the same polar representation the lidar already uses, so that the identical Bresenham ray-tracing and log-odds update logic can fuse them into the map. They are written into a dedicated, camera-only obstacle layer rather than into the shared lidar wall map, specifically so that the lidar's own routine free-space sweeps, which pass straight through the obstacle at their own fixed height, can never accidentally erase a low obstacle only the camera can see. Two further safety checks guard this layer's quality: a candidate is rejected outright if the straight line from the camera to it would have to pass through a cell already confirmed solid, by the lidar or by an earlier camera detection, since a camera physically cannot see through solid matter; and a candidate is rejected if its registered colour matches the tracked blue or yellow target object, since that case is already handled more thoroughly by the dedicated colour-tracking pipeline described above.

Finally, because the camera only ever observes the near-facing side of an obstacle, with everything behind it necessarily occluded and therefore unknown, every accepted hit is extended a short additional distance further along the same ray to fill in a plausible occluded body behind the visible face. Without this padding, the planner would be free to route the robot straight through an obstacle's unseen far side. The padding itself remains ordinary, correctable log-odds evidence: it stops as soon as it would reach an already-confirmed wall, rather than being extended blindly through it, and a later viewpoint that genuinely sees clear space through those cells erodes the padding again in the normal way.

7. Reactive Safety Fallback (Distance Sensors)

While the robot navigates using its planned path, a short-range reflex layer runs independently of the main exploration logic and takes priority over it at every control step. This is necessary because the path planning relies entirely on the occupancy map, which can be inaccurate due to odometry drift or obstacles that were never mapped — for instance, low or thin objects that fall outside the lidar's fixed scanning height. The four infrared distance sensors mounted on the robot observe the environment directly, without going through the mapping pipeline, and therefore serve as a last line of defense against collisions. If one of the front sensors detects an obstacle closer than a defined safety threshold, the robot immediately halts its current behavior and performs a short evasive maneuver: the detected obstacle is written into the occupancy map so that future path planning routes around it, the robot reverses briefly while steering slightly away from the side where the obstacle was closer, and a full replanning cycle is triggered afterward. For the duration of this maneuver, the reflex remains active and overrides the rest of the control logic until the backup phase has finished.

8. Stuck Detection

Separate from the reactive collision avoidance, the system continuously monitors whether the robot is making actual progress toward its current target, since a path can appear valid while the robot is effectively unable to follow it — for example when it gets caught against a wall or an unmapped obstacle. Two indicators are used to detect this condition. The primary one tracks the robot's position over time and flags it as stuck if it has not moved a minimum distance within a defined time window. A second, faster-acting indicator uses the inertial measurement unit to detect whether the robot has physically tipped over; since this situation

can otherwise persist for a while before the distance-based check notices it, it is treated with higher priority whenever it occurs. Once either condition is detected, the current target is marked as temporarily unreachable so that the same failing goal is not immediately chosen again, the current path is discarded, and the robot performs a brief backward maneuver before replanning from its new position. This exclusion list of temporarily unreachable targets is not permanent; it is cleared periodically after a number of successful planning cycles so that targets which have become reachable again — for example after the map was updated with new information — get a fair chance to be selected once more.

9. Replanning Logic

Replanning forms the core adaptive mechanism that keeps the robot's behavior consistent with the most recent state of the environment. Rather than committing to a single plan until it either succeeds or fails outright, the system re-evaluates its current path under several independent conditions, each addressing a different reason the existing plan might no longer be appropriate. The most straightforward trigger occurs once the current path has been fully completed or the target has been reached, at which point a new goal must be selected. Beyond this, replanning is also triggered periodically after a fixed time interval, regardless of whether the current path still appears valid — this reflects the fact that the map is continuously updated as the robot moves, so a plan computed even a short time earlier may already be based on outdated information, and a better route or target may have since become available. A further trigger occurs when newly gathered sensor data reveals that a cell along the currently planned path has become blocked, since continuing to follow an invalidated path serves no purpose. Finally, the stuck- and tip-over-detection mechanisms described above also force a replanning cycle, but combined with a short backward maneuver first, since replanning immediately from the same stuck position would likely reproduce the same unsuccessful plan.

A special case arises when the robot is heading toward a known target object rather than an unexplored frontier, since the object's exact position is fixed and cannot simply be exchanged for an alternative like a frontier can. If a direct path to the object cannot be found, this is usually not because no physical route exists, but because the safety margin kept around mapped obstacles has sealed off the only way through — for instance when the object sits close to a wall. In this situation, the system falls back to planning a path toward the nearest point it can safely reach at the edge of that safety margin, effectively parking as close to the object as possible without violating the required clearance, instead of giving up entirely. Should even this fallback plan fail to make any real progress — for example because the reachable edge point coincides with the robot's current position, which would otherwise cause the same ineffective plan to be recomputed indefinitely — the safety margin is temporarily reduced by one grid cell and the planning attempt is repeated once, which is often enough to open up a narrow gap that the full margin had sealed off. Together, these mechanisms allow the robot to follow a chosen path efficiently under normal conditions while remaining responsive to new information, recovering automatically from situations where the original plan is no longer valid, and still making a reasonable final approach even when a target cannot be reached with full safety clearance.